CS 681
Programming Assignment 2

Neel Parekh and Kamal Charotia
25M0791 and 25M0814
Shared-Data

Simulation Design

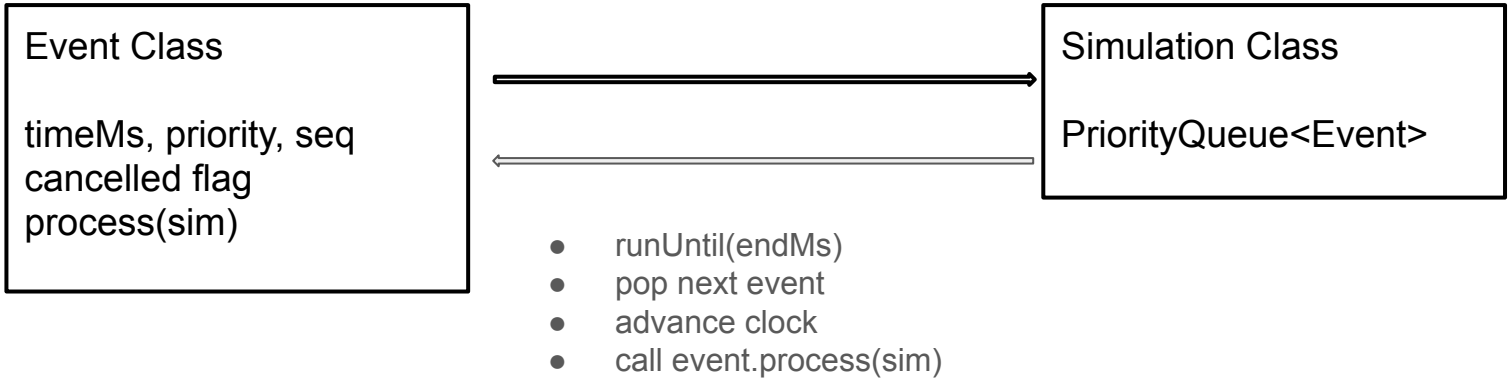
Welch mode: loops replications, aggregates bins
Experiment Mode: user sweep, replications and writes csvs.

Simulator will build one replication end-to-end:

- Simulation kernel
- Server / Core / Thread / Users
- Distributions + RNG streams
- Metrics + Trace + optional WelchCollector

Simstate(passed to all events): thinkDist, serviceDist, timeoutDist, server, users, metrics, trace, welchCollector.

Basic Design Structure



Call Graph

run_experiment.sh → Main.java

Main.java —Mode = Experiment→ ExperimentRunner.java

ExperimentRunner.java

- user sweep + replications
- writes summary.csv
- writes replications.csv

ExperimentRunner.java —Pass all configuration to run experiment→ Simulator.java

Simulator.java

Build one replication end-to-end:

- Simulation kernel
- Server / Core / Thread / Users
- Distributions + RNG streams
- Metrics + Trace

Events Handlers

1. UserIssueRequestEvent
   a. sample cpu + timeout
   b. create Request
   c. schedule RequestTimeoutEvent
   d. server.accept(request): dispatch OR queue OR drop
2. CoreSliceEndEvent
   a. finish one quantum on a core/thread
   b. if remainingCpu <= 0: complete request
      i. not timed out -> good completion + user schedules next
      ii. timed out -> bad completion
   c. else: re-enqueue thread for RR
3. RequestTimeoutEvent
   a. if request not completed: mark timedOut, metrics++, user unblocked and thinks
4. WarmupResetEvent
   a. start measurement window, reset queue/util accounting

Discrete Event Simulation - Setup 1
Behavior of M/G/c system in Round-Robin Scheduling?

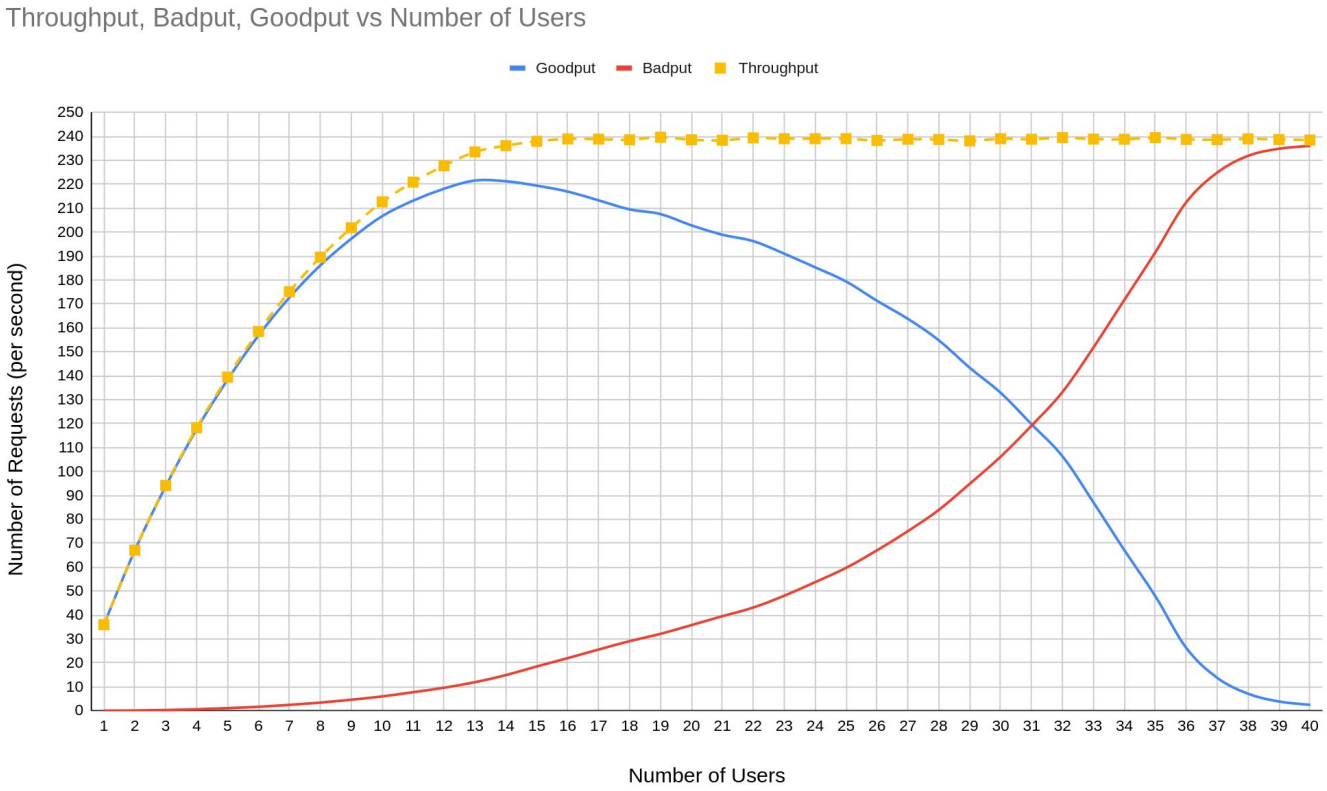
Simulation Configuration:

- Number of cores: 4
- Number of threads : 16
- Number of users: [1…40]
- Think time: Triangular(5ms, 10ms, 20ms)
- Timeout: Uniform(80ms, 200ms)
- Queue length: 50
- Scheduling Policy: Round Robin
- Quantum: 5ms
- Context switch overhead: 0.2ms (4% of quantum)
- Warmup time: 2s (Max. time found through Welch procedure)

Experiment methodology:

- Closed load testing only
- 5 runs with different seeds of experiment simulated for each user value from from 1 to 40.
- Simulated for 5 min of actual experiment.

Throughput, Goodput, Badput vs Number of Users



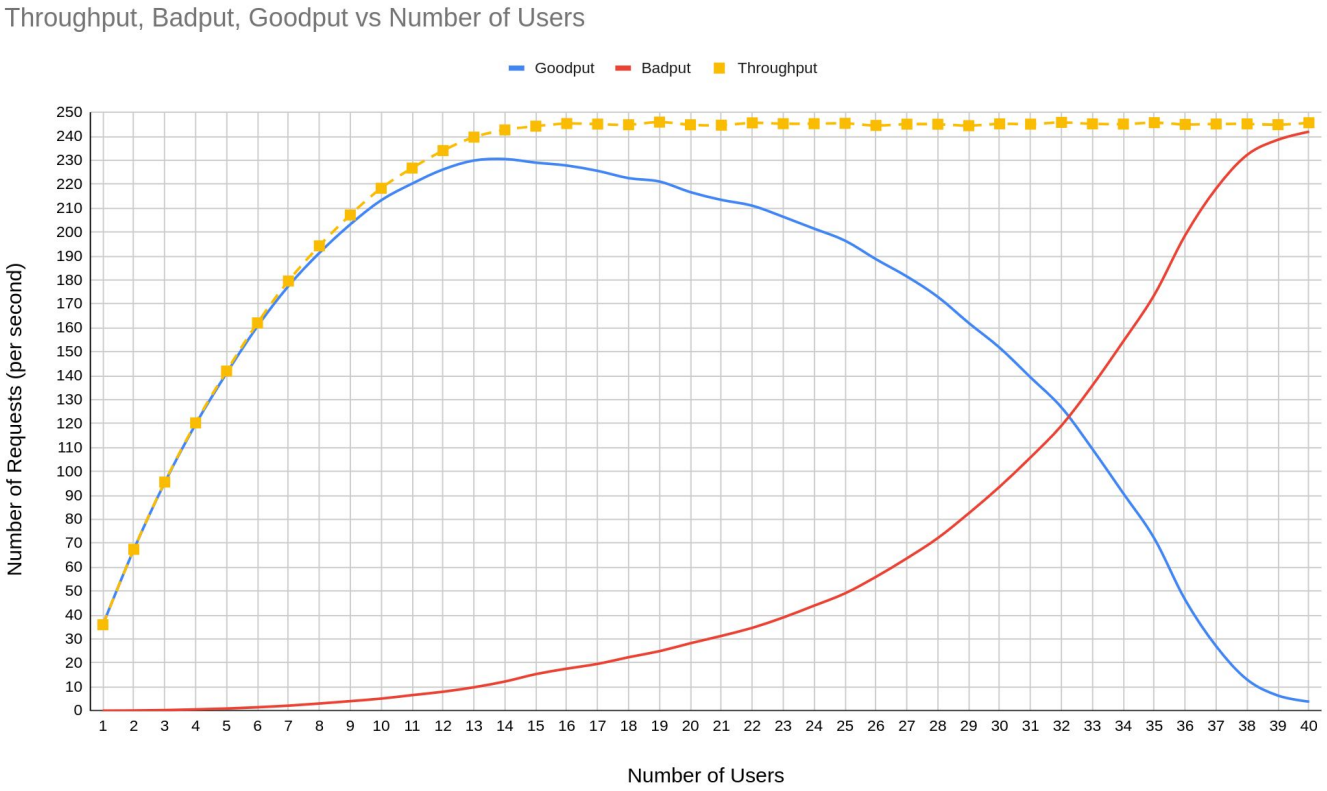
Timeout
Uniform(80ms, 200ms)

Service Time Distribution
Shifted(2ms, Exponential(14ms))

Observations

1. Throughput of the system: 240 Requests per second
2. Maximum number of users (M*): ~14 Users
3. Maximum goodput: ~220 Requests per second @ 13 Users
   a. This is less than system throughput, indicating that request are getting timed out while waiting in the queue.
   b. (Hypothesis) We may increase quantum of round-robin to allow lesser response time of requests.
4. At 31 users, system goodput goes below system badput, indicating that majority of requests are timing out while in queue.
5. At 40 users, badput, approaches system throughput, indicating all requests are timing out while in queue.

Throughput, Goodput, Badput vs Number of Users



Timeout
Uniform(80ms, 200ms)

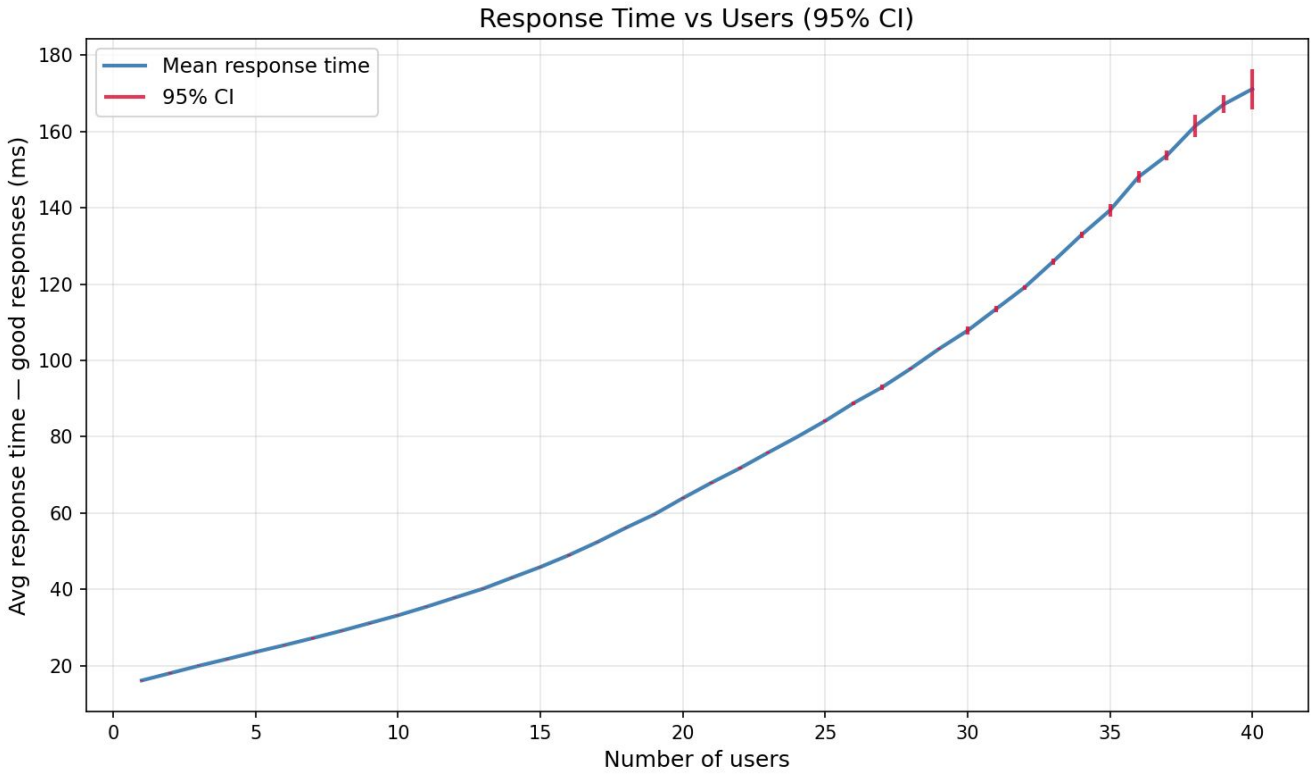
Service Time Distribution
Shifted(2ms, Exponential(14ms))

Quantum (Changed)
16ms

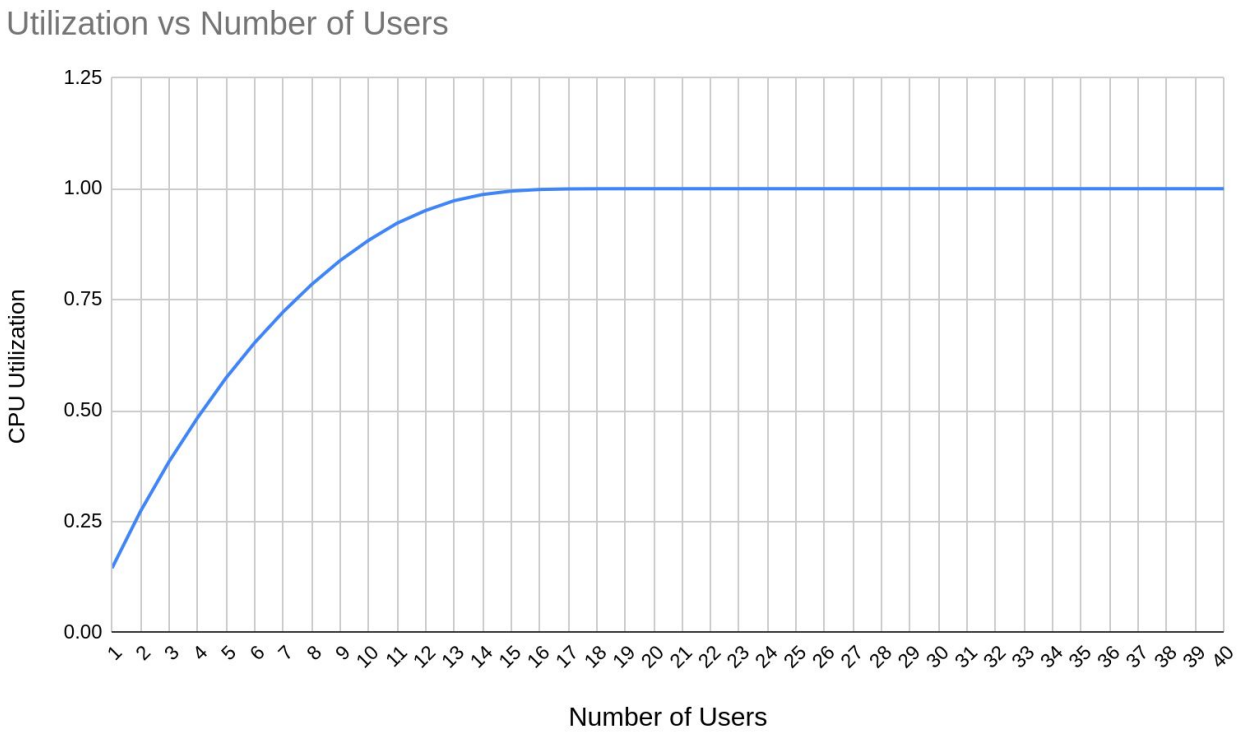
Observations (Verification of Hypothesis)

1. Only quantum of round robin has been increased while keeping all other parameters constant.
2. System throughput and max number of users are same as before.
3. Maximum goodput starts to drop @ ~14 Users which is M*
   a. With increased quantum of round robin, each request waiting time decreases.
   b. Service time distribution and quantum time are similar at 16ms.
   c. As service time of request on average approaches to system service time, goodput drops after M*.

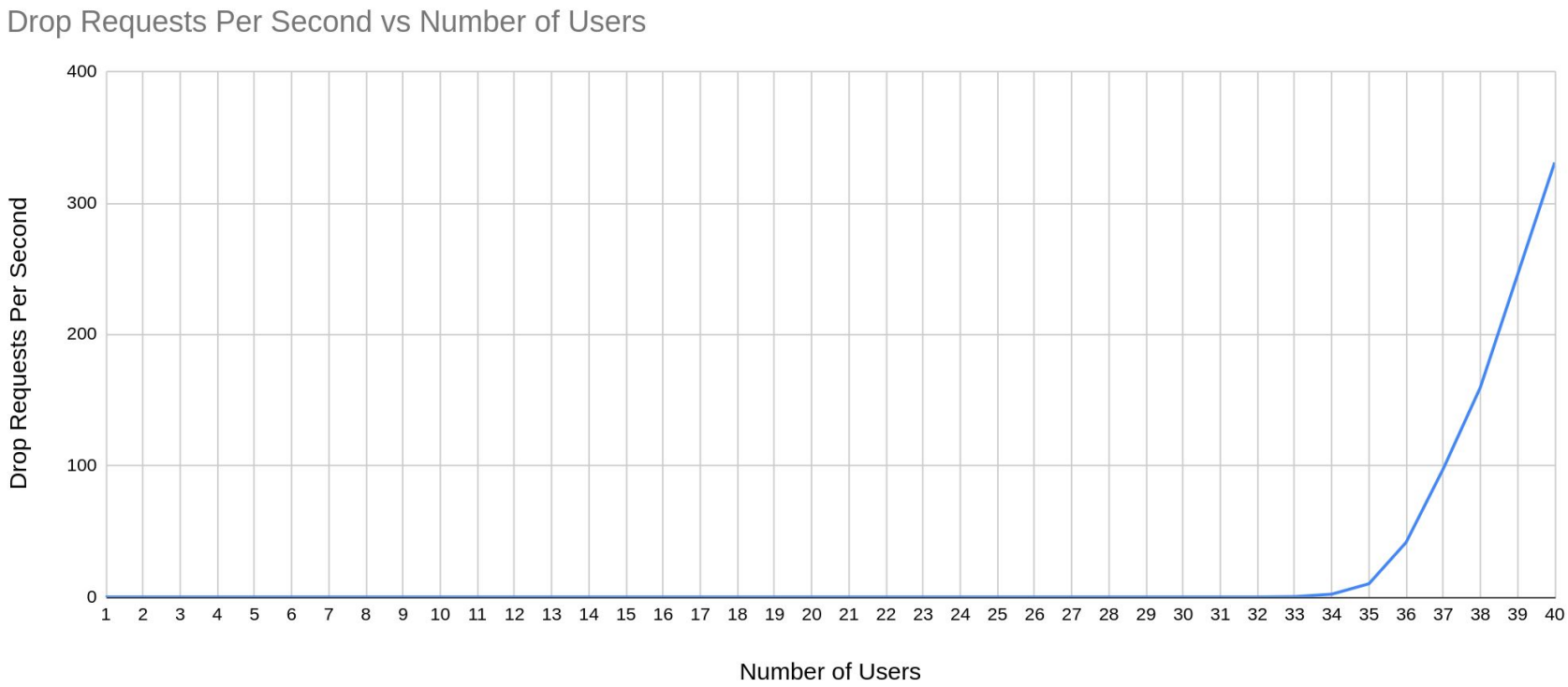
Response Time vs Number of Users



Utilization vs Number of Users



Drop Requests vs Number of Users



Observations

1. Response time is monotonically increasing for all users.
2. Unlike FCFS scheme of scheduling, Round-Robin scheme seems not to allow constant response time when throughput is less than system throughput.
3. Reason for this:
   a. Round-robin scheduling is slicing requests
   b. Each request is facing more waiting time in queue due to slicing of time quantum.
   c. As number of users increases in system, each request on average has to wait for more number of quantum
   d. In FCFS only initial waiting time on receiving request was included in response time
   e. In round-robin waiting time after each suspension is also being included which increases with increasing number of users.
   f. Though slope is flat compared to slope of graph after M*.
4. Utilization Graph:
   a. Graph is linearly increases till server saturation of 14 users and thereafter flattens out as system is fully saturated.

Discrete Event Simulation - Setup 2
Effect of Queue Size on system

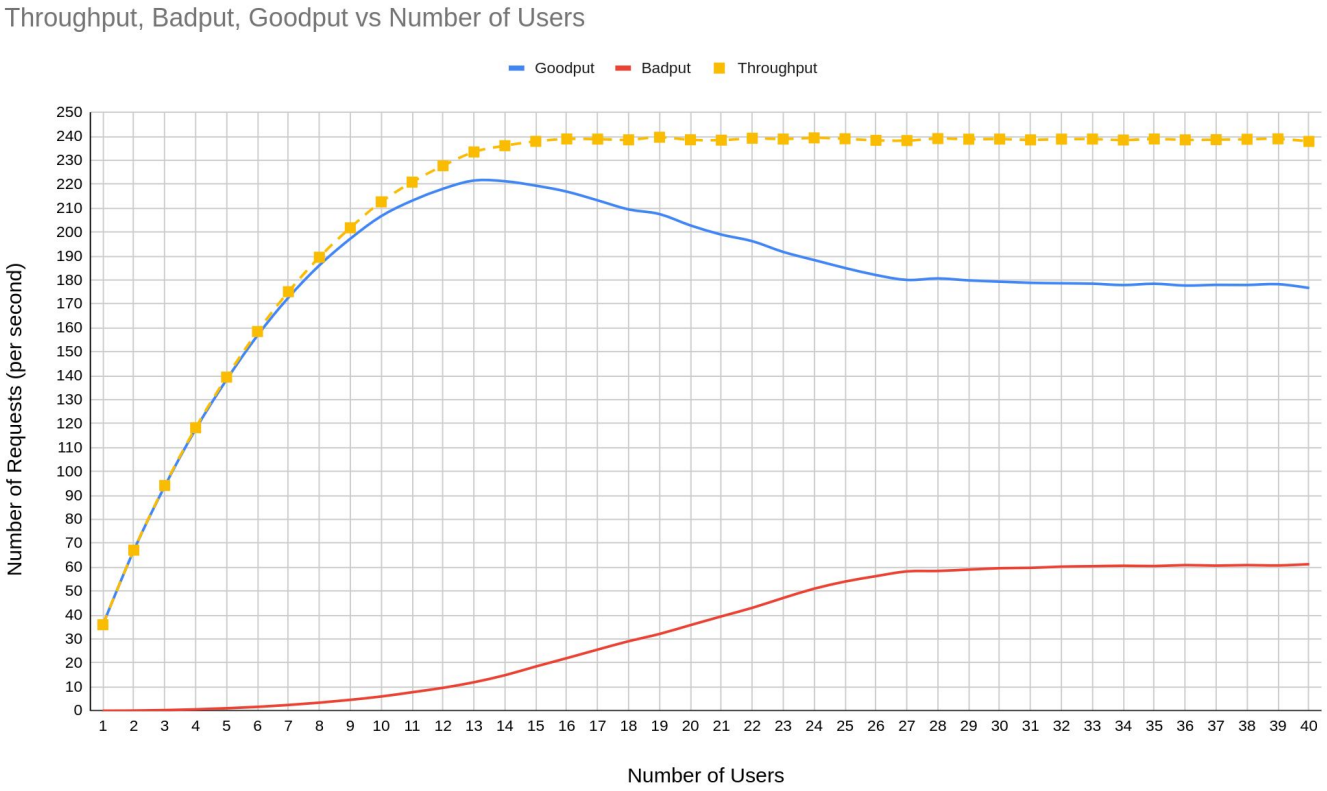
Simulation Configuration:

- Number of cores: 4
- Number of threads : 16
- Number of users: [1…40]
- Think time: Triangular(5ms, 10ms, 20ms)
- Timeout: Uniform(80ms, 200ms)
- **Queue length: 10 (Earlier 50)**
- Scheduling Policy: Round Robin
- Quantum: 5ms
- Context switch overhead: 0.2ms (4% of quantum)
- Warmup time: 2s (Max. time found through Welch procedure)

Experiment methodology:

- Closed load testing only
- 5 runs with different seeds of experiment simulated for each user value from from 1 to 40.
- Simulated for 5 min of actual experiment.

Throughput, Goodput, Badput vs Number of Users

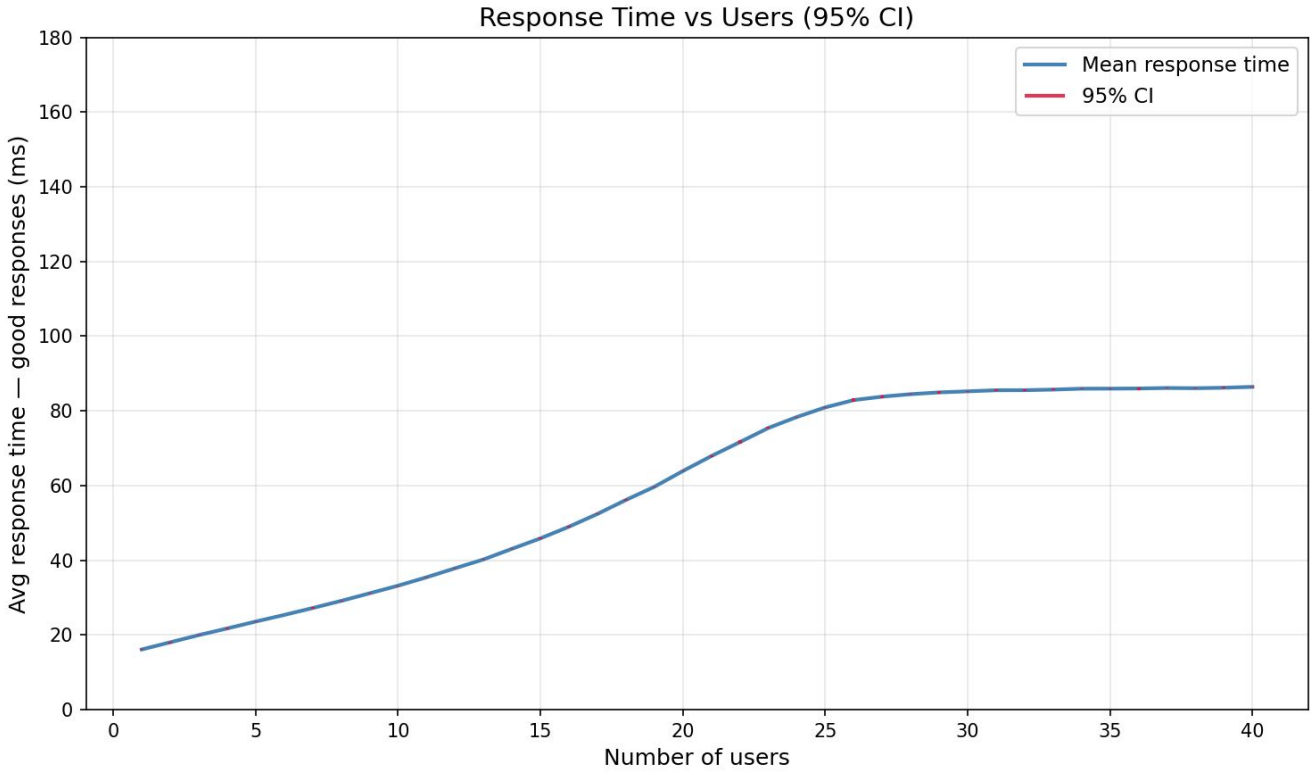


Timeout
Uniform(80ms, 200ms)

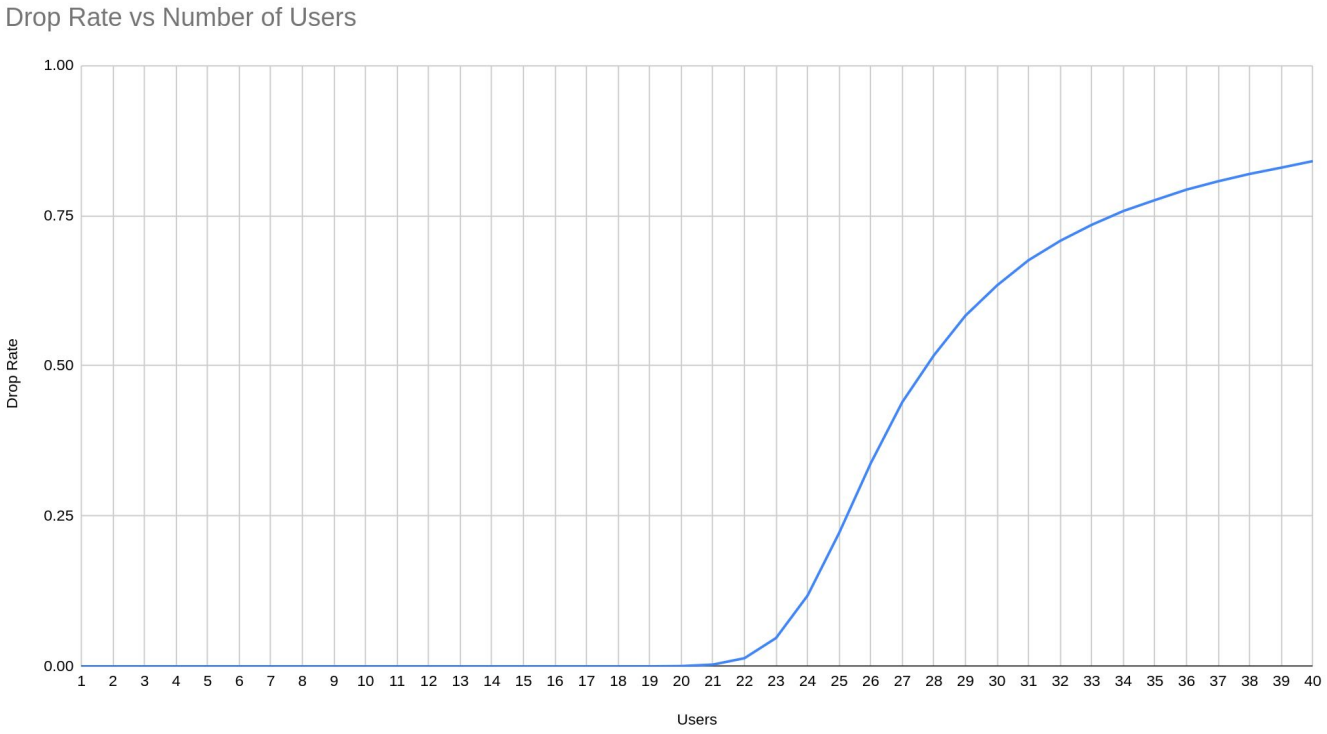
Service Time Distribution
Shifted(2ms, Exponential(14ms))

Queue Length
10

Response Time vs Number of Users



Drop Requests vs Number of Users



Observations

1. Badput and Goodput never crosses like earlier
2. Possible reason:
    a. In response time graph, it flattens after 26 users even in round robin scheduling.
    b. Therefore system has limited capacity to hold requests in queue which saturates at 26 users
    c. After 26 users, including any more users must lead to request drops
    d. Same is indicated by Drop Request graph which starts to increase at around 23 users
    e. Since number of requests in system saturates, the goodput and badput must approach to some constant
    f. Same is suggested by throughput graph.

In earlier setup, queue length was 50 which was not saturated by 40 users. This allowed more requests to be in system which could add to badput and goodput. But with queue 10, system is saturated well before badput could get more than goodput.

Discrete Event Simulation - Setup 3
Constant Service Time

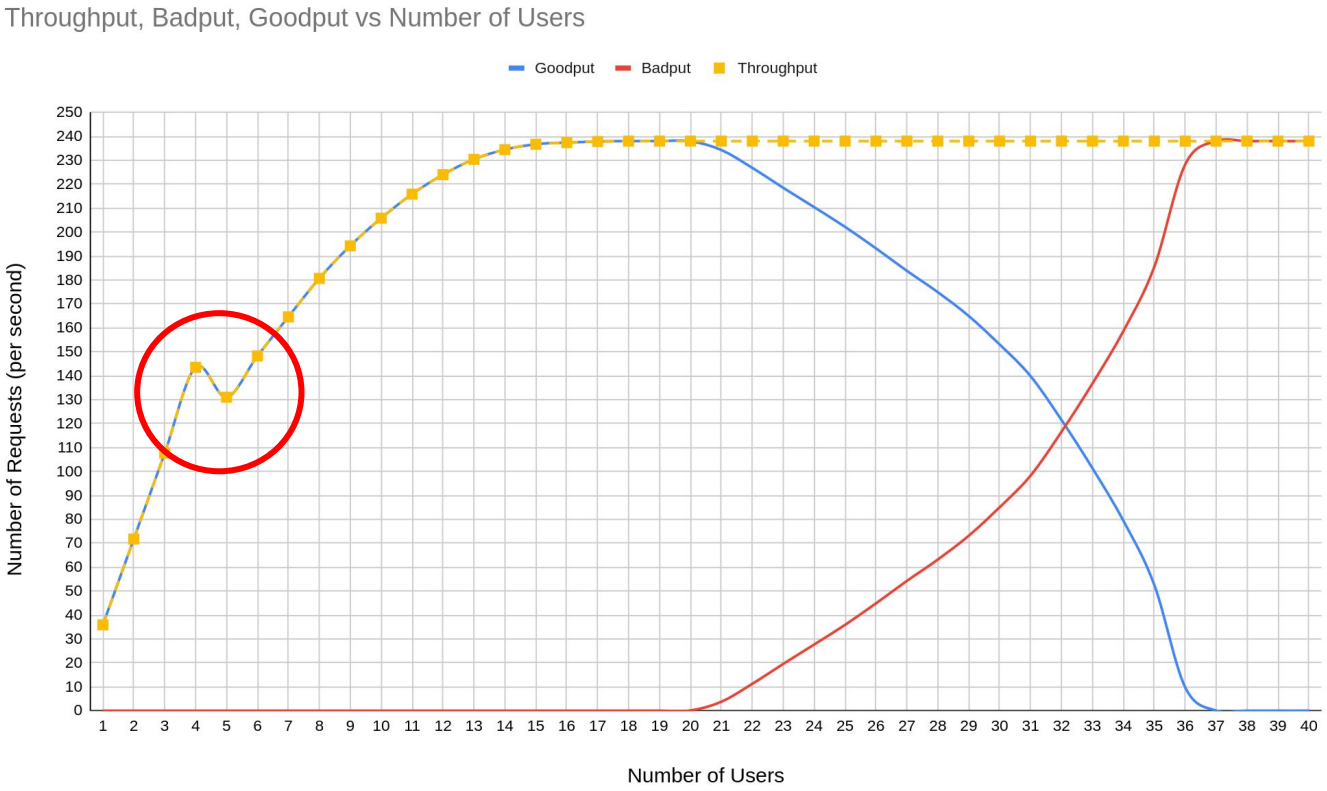
Simulation Configuration:

- Number of cores: 4
- Number of threads : 16
- Number of users: [1…40]
- Think time: Triangular(5ms, 10ms, 20ms)
- Timeout: Uniform(80ms, 200ms)
- Queue length: 50
- Scheduling Policy: Round Robin
- Quantum: 5ms
- Context switch overhead: 0.2ms (4% of quantum)
- Warmup time: 2s (Max. time found through Welch procedure)

Experiment methodology:

- Closed load testing only
- 5 runs with different seeds of experiment simulated for each user value from from 1 to 40.
- Simulated for 5 min of actual experiment.

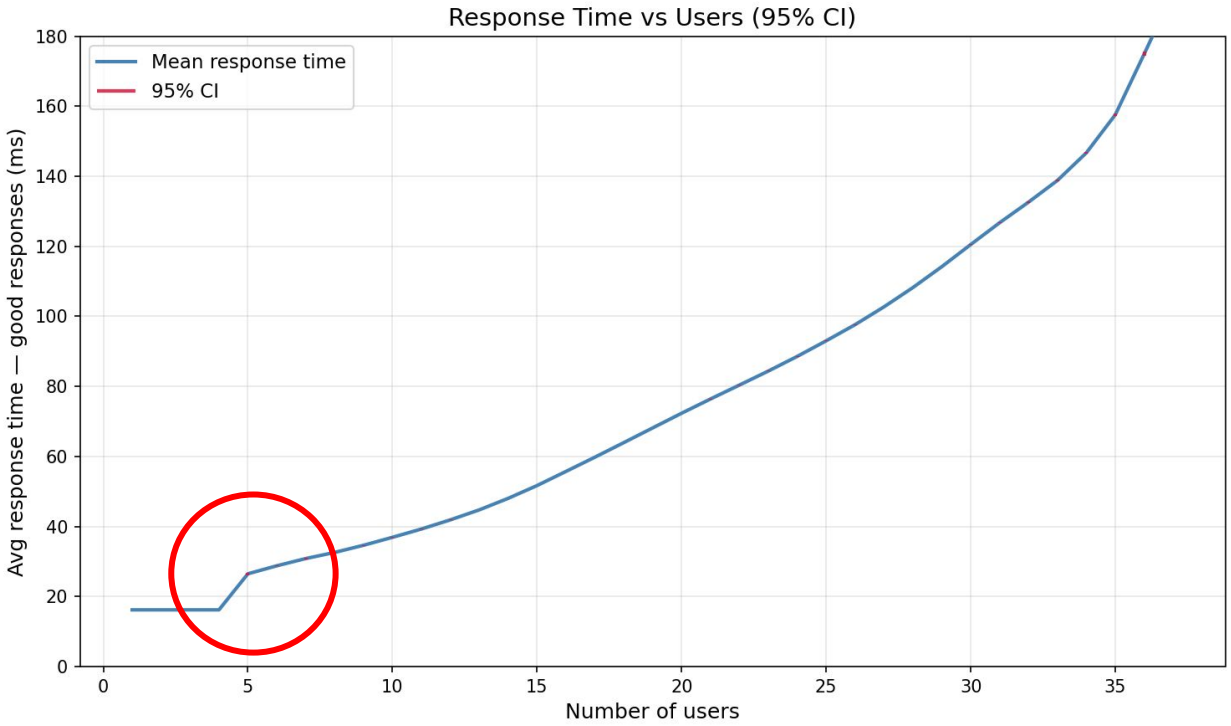
Throughput, Goodput, Badput vs Number of Users



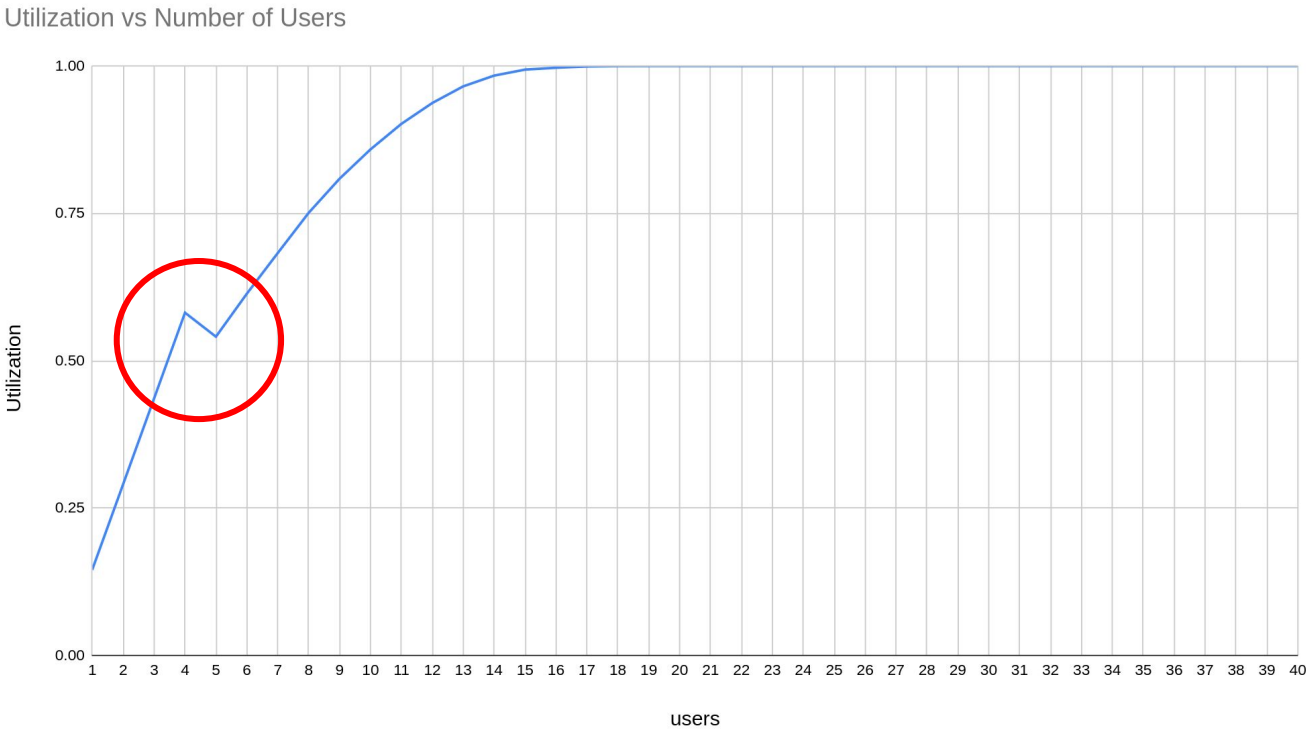
Timeout
Uniform(80ms, 200ms)

Service Time Distribution
Constant 16ms

Response Time vs Number of Users



Utilization vs Number of Users



Observations

In above three graph highlighted area suggests peculiar activity.

1. Utilization graph shows decrease in utilization starting @ 5 users.
2. Same activity is present in other 2 graphs as well. Showing slight drop in activity at 5 users.
3. Simulation done for 4 CPU core
4. Activity has been verified with other constant service time value.
5. It is correlated to number of CPU for which simulation is performed
6. It happens in round-robin scheduling

We are unable to explain the mechanism of this activity